

Identity and Authorization Across Domains

A summary of the environment flow and the core problem

Environment Flow

The architecture utilizes a central API Gateway that manages external traffic for two completely isolated identity clusters.

1. **System A Flow:** "System A Users" access the environment via the external address `rhssso.apps.cluster.domain.com`. The API Gateway routes this traffic to the internal address of the legacy RH-SSO 7.6.5 cluster (deployed on 3 VMs), which hosts `client-a`.
2. **System B Flow:** "System B Users" access the environment via the external address `rhbk.apps.cluster.domain.com`. The API Gateway routes this traffic to the internal address of the new RHBK 26.4 cluster (deployed on OpenShift), which hosts `client-b`.
3. **Cross-Service Execution:** The execution flow occurs when `client-b` authenticates with the RHBK cluster, obtains a token, and its backend attempts to make an API call to the backend of `client-a` (which still resides in the legacy environment) using that new token.

The Problem When `client-b` presents the RHBK-issued token to `client-a`, the transaction is rejected with an `invalid token` error.

Although both clusters federate the same Active Directory for user credentials, they do not share cryptographic trust. Because they are isolated behind the API Gateway, they possess entirely different RSA signing keys and different issuer URLs. When `client-a` tries to validate the token's signature and issuer string using its local RH-SSO configuration, the validation fails. The old system does not recognize the new system's tokens, breaking service-to-service communication during this transition state.

Possible solution

Disclaimer: The solution, based solely on official documentation. I have not tested it in any environment.

Instead of forcing **client-a** (on the old RH-SSO cluster) to understand tokens from the new RHBK cluster, you use the JWT Authorization Grant to perform "Identity and Authorization Chaining Across Domains".

1. **The Trust Link:** You configure the old RH-SSO cluster to treat the new RHBK cluster as a trusted external Identity Provider.
2. **The Exchange:** When **client-b** obtains a token from the new RHBK cluster, it does not send it directly to **client-a**. Instead, **client-b** (or a centralized backend service) sends a request to the old RH-SSO token endpoint via the API Gateway (rhssso.apps.cluster.domain.com), presenting the RHBK token as a client assertion.
3. **The Validation:** The RH-SSO server automatically reaches out (via the Gateway) to fetch RHBK's public keys from its JWKS endpoint. RH-SSO validates the signature and claims of the RHBK token.
4. **The New Token:** Once validated, RH-SSO issues a brand new access token native to the old cluster.
5. **The API Call:** **client-b** uses this newly minted RH-SSO token to call **client-a**.

Why this solution?

- **Zero Changes to Legacy Apps:** **client-a** receives a standard token issued by its own familiar RH-SSO server. It requires zero code changes, no multi-issuer logic, and no new environment variables.
- **Centralized Security:** The trust relationship is managed strictly between the two Identity and Access Management (IAM) servers. Your microservices remain completely oblivious to the migration.
- **Standards-Based:** This utilizes the IETF standard RFC 7523, meaning it is cryptographically secure and natively supported by Keycloak/RHBK without needing custom plugins.

Implementation Overview

To implement this, you must configure the old RH-SSO server:

1. Add the new RHBK cluster as an OpenID Connect Identity Provider.
2. In the RH-SSO Admin Console, navigate to the specific client that will request the exchange.
3. Under the **Capability config** section, enable the **JWT Authorization Grant** capability.
4. In the "Allowed Identity Providers for JWT Authorization Grant" option, select the RHBK Identity Provider you just created.

Critical Security Risk for the Environment (RHBK 26.4)

While this is the correct architectural path, you must be aware of a specific vulnerability present in your target version. There is an active flaw (CVE-2026-1486) in Keycloak/RHBK 26.4 specifically regarding the `jwt-authorization-grant` flow.

The server currently fails to verify if an Identity Provider (IdP) is actively enabled before issuing tokens during this grant. This means if you ever disable the RHBK Identity Provider connection inside RH-SSO (for example, if a compromise is suspected), the server will still accept valid JWT assertions signed by that provider and issue access tokens. You must ensure strict key rotation and management policies are in place to mitigate this until the environment is fully migrated or patched.

References:

<https://www.keycloak.org/securing-apps/jwt-authorization-grant>

<https://www.keycloak.org/securing-apps/oauth-identity-authorization-chaining-across-domains>

The API Gateway

The API Gateway is utilized in this solution in three critical ways:

1. **Facilitating the Token Exchange:** When `client-b` (located in the new environment) needs to swap its token, it cannot route directly to the old RH-SSO server. It must send its JWT Authorization Grant request out through the API Gateway using the external routing address (`rhssso.apps.cluster.domain.com`).
2. **Server-to-Server Key Fetching (JWKS):** During the exchange, the old RH-SSO server must mathematically validate the assertion token sent by `client-b`. To do this, RH-SSO needs the public keys from RHBK. Because they are internally isolated, the RH-SSO server must make an outbound HTTP request through the API Gateway using the new external address (`rhbk.apps.cluster.domain.com`) to download the keys from RHBK's JWKS endpoint.
3. **Preserving Issuer Integrity:** By forcing all cross-cluster trust requests back through the API Gateway's external addresses, the system ensures that the `iss` (issuer) strings inside the tokens perfectly match the external hostnames the servers expect to see. If they attempted to communicate using internal IP addresses, the token validation would fail instantly due to an issuer mismatch.